National University of Computer and Emerging Sciences



Lab Manual 01
Artificial Intelligence

Course Instructor	Ms. Bushra Rashid
Lab Instructor	Muhammad Hasan
Lab Instructor Email ID	m.hasan@lhr.nu.edu.pk
Semester	FALL 2026

FAST - School of Computing
FAST-NU, Lahore Campus

Contents

Objectives: 4

Activities: 4

Activity 1 (Anaconda Navigator – Jupyter Notebook): 4

Activity 2 (Python): 4

1. Data types: 4
2. Typecasting: 5
3. Operators: 6
   a. Arithmetic Operators: 6
   b. Comparison Operators: 6
   c. Boolean Operators 7
      i. and Operator: 7
      ii. or Operator: 7
      iii. not Operator: 7
4. Conditional Execution: 7
   a. Simplest Conditional Execution: 7
   b. Alternative Conditional Execution: 8
   c. Chained Conditional Execution: 8
   d. Nested Conditional Execution: 8
5. Loops and Iterations 9
   a. Repeated Steps 9
   b. An Infinite Loop 9
   c. Breaking Out of a Loop 9
   d. Finishing an Iteration with Continue 9
   e. A Simple Definite ‘for’ Loop 10
   f. A Definite ‘for’ Loop with Strings 10
   g. A ‘for’ Loop with range function 10
6. Functions: 10
   a. Build-in Functions 10
   b. Custom Functions 11

Activity 3 (Examples): 11

Stripping Whitespace 11

Filtering lines/input with startswith() function 11

Strings are not Mutable 11

Looking Inside Lists 12

How Long is a List? 12

Concatenating Lists Using + ... 12
Lists Can Be Sliced Using : 12
Is Something in a List? 12
Dictionaries 13
Comparing Lists and Dictionaries 13
Dictionary Literals (Constants) 13
Many Counters with a Dictionary 13
Dictionary Tracebacks 13
Checking key in dictionary 14
When We See a New key in dictionary 14
The get Method for Dictionaries 14
Retrieving lists of Keys and Values 15
Definite Loops and Dictionaries 15
Activity 4 (Problem Statements / Tasks): 15
1. Student Performance Analyzer 15
2. Comment-Aware Word Frequency Analyzer 16
3. Shopping Cart & Billing System 16
4. Data Cleaning & Statistical Summary 17
5. User Login Validation System 17

Objectives:

- ✓ Introduction to Python Interpreter
- ✓ Introduction to Python
    - Data types
    - Operators
    - Basic I/O
    - Conditional Execution / Selection Statement
    - Loops / Iterations
    - Functions

Activities:

Activity 1 (Anaconda Navigator – Jupyter Notebook):

Introduction, Installation and use of Anaconda Navigator – Jupyter Notebook is demonstrated by instructor.
For more information: https://learncodingfast.com/how-to-install-anaconda-and-use-jupyter-notebook/

Activity 2 (Python):

1. *Data types:*

Sr#	Categories	Data Type	Examples
1	Numeric Types	int	-2, -1, 0, 1, 2, 3, 4, 5
2		float	-1.25, -1.0, --0.5, 0.0, 0.5, 1.0, 1.25
3		complex	3 + 4j
4	Text Sequence Type	str	'a', 'Hello!'
5	Boolean Type	bool	True, False
6	Sequence Types	list	["apple", "banana", "cherry"]
7		tuple	("apple", "banana", "cherry")
8		range	range(6) → 0 to 5
9	Mapping Type	dict	{"name" : "John", "age" : 36}
10	Set Types	set	{"apple", "banana", "cherry"}
11		frozenset	frozenset({"apple", "banana", "cherry"})

Note:

- ✓ Python has no command for declaring a variable for any data type. A variable is created the moment you first assign a value to it.

- ✓ Variable names are case-sensitive.
- ✓ Strings are "immutable" (Unchangeable).
- ✓ List is a collection which is ordered and changeable. Allows duplicate members.
- ✓ A list element can be any Python object - even another list. A list can be empty.
- ✓ Tuple is a collection which is ordered and unchangeable. Allows duplicate members.
- ✓ The range function returns a list of numbers that ranges from zero to one less than the parameter.
- ✓ Dictionary is a collection which contains key & value pairs, is ordered (as of Python 3.7 and later) and changeable. No duplicate members (keys).
- ✓ Set is a collection which is unordered, unchangeable (but you can remove items and add new items), and unindexed. No duplicate members.

2. *Typecasting:*

The process of converting the value of one data type to another data type is called type casting. In type casting, loss of data may occur as we enforce the object to a specific data type.

```
# cast to float
x = float(2)
y = float(30.0)
z = float("20")
print(x)
print(y)
print(z)

# cast to str
x = str(2)
y = str(30.0)
z = str("20")
print(x)
print(y)
print(z)

# cast float to int and string
f = 100.05

n = int(f)
print(n)
print(type(n))

s = str(f)
print(s)
print(type(s))


# Sum two numbers using typecast
num_int = 123
num_str = "456"
print("Data type of num_int:",type(num_int))
print("Data type of num_str before Type Casting:",type(num_str))
num_str = int(num_str)
```

```
print("Data type of num_str after Type Casting:",type(num_str))
num_sum = num_int + num_str
print("Sum of num_int and num_str:",num_sum)
print("Data type of the sum:",type(num_sum))
```

3. *Operators:*

a. Arithmetic Operators:

Operators	Operation	Examples
**	Exponent	2 ** 3 = 8
%	Modulus/Remainder	22 % 8 = 6
//	Integer division	22 // 8 = 2
/	Division	22 / 8 = 2.75
*	Multiplication	3 * 3 = 9
-	Subtraction	5 - 2 = 3
+	Addition	2 + 2 = 4

b. Comparison Operators:

Operators	Meaning	Examples
==	Equal to	2 == 2 → True
!=	Not equal to	2 != 2 → False
<	Less than	5 < 1 → False
>	Greater Than	5 > 1 → True
<=	Less than or Equal to	3 < 5 → True
>=	Greater than or Equal to	5 >= 5 → True

c. Boolean Operators

i. and Operator:

Expression	Evaluates to
True and True	True
True and False	False
False and True	False
False and False	False

ii. or Operator:

Expression	Evaluates to
True or True	True
True or False	True
False or True	True
False or False	False

iii. not Operator:

Expression	Evaluates to
not True	False
not False	True

4. Conditional Execution:

a. Simplest Conditional Execution:

```
x = 5
y = 2

# Single statements in if block
if x > 0 :
  print('x is positive')
```

```
# multiple statements in if block
# Python uses indentation instead of curly-brackets
# to define the scope in the code.
if x > y :
  print(x)
  print(y)
```

b. Alternative Conditional Execution:

```
x = 7

if x % 2 == 0 :
  print('x is even')
else :
  print('x is odd')
```

c. Chained Conditional Execution:

```
x = 5
y = 7

if x < y :
  print('x is less than y')
elif x > y :
  print('x is greater than y')
else :
  print('x and y are equal')

# else clause is not compulsory
# but if there is an else clause, it has to be at the end
```

d. Nested Conditional Execution:

```
x = 7
y = 8

if x != y :
  if x < y :
    print('x is less than y')
  else :
    print('x is greater than y')
else :
  print('x and y are equal')
```

5. *Loops and Iterations*

a. Repeated Steps

```
n = 5
while n > 0 :
  print(n)
  n = n - 1

print('Blastoff!')
print(n)
```

b. An Infinite Loop

```
n = 5
while n > 0 :
  print('Lather')
  print('Rinse')

print('Dry off!')
```

c. Breaking Out of a Loop

```
while True:
  line = input('Enter name: ')

  if line == 'done' :
    break

  print(line)


print('Done!')
```

d. Finishing an Iteration with Continue

```
while True:
  line = input('> ')

  if line[0] == '#' :
    continue

  if line == 'done' :
    break

  print(line)

print('Done!')
```

e. A Simple Definite 'for' Loop

```
for i in [5, 4, 3, 2, 1] :  # here we have the list of 5 integers
  print(i)

print('Blastoff!')
```

f. A Definite 'for' Loop with Strings

```
friends = ['Ali', 'Ahmad', 'Mustafa']

for friend in friends :
  print('Happy New Year:', friend)

print('Done!')
```

g. A 'for' Loop with range function

```
for i in range(5):
    print('Current Value of i is:', i)

# The range() function can also be called with three arguments.
# The first two arguments will be the start and stop values,
# and the third will be the step argument. The step is the amount
# that the variable is increased by after each iteration.
```

6. *Functions:*

a. Build-in Functions

```
# print function
print("Hi There")

name = "Ali"
print('Hello {}'.format(name))

name = input('Enter name: ')
age = int(input('Enter age: '))
print(f"My name is {name} and I am {age} years old.")

print('Hands-on','python','programming','lab',sep='\n')

# abs integer number
num = -5
print('Absolute value of -5 is:', abs(num))

# abs floating number
fnum = -1.45
print('Absolute value of 1.45 is:', abs(fnum))
```

```
# max function
number = [3, 2, 8, 5, 10, 6]
largest_number = max(number);
print("The largest number is:", largest_number)

# sum function
my_list = [1,3,5,2,4]
print("The sum of my_list is", sum(my_list))
```

Refer to this link for the complete list of Python built-in functions.

b. Custom Functions

```
def hello(name):
     print('Hello {}'.format(name))

hello('Alice') #Hello Alice
hello('Bob') #Hello Bob

def sum1(num1, num2):
  return num1 + num2

print(sum1(7, 8))
```

Activity 3 (Examples):

Stripping Whitespace

```
greet = ' Hello Bob '
print("'" + greet.lstrip() + "'")
print("'" + greet + "'")
print("'" + greet.rstrip() + "'")
print("'" + greet.strip() + "'")
```

Filtering lines/input with startswith() function

```
line = 'Please have a nice day'
print(line.startswith('Please'))
print(line.startswith('P'))
print(line.startswith('p'))
```

Strings are not Mutable

```
fruit = 'Banana'
print(fruit)
fruit = 'Apple'
print(fruit)
fruit[0] = 'b'
```

```
fruit = 'Banana'
x = fruit.lower()       # it creates the copy of lower case
                        # instead of chagning in original string
print(fruit)
print(x)
# For more string methods:
# https://docs.python.org/3/library/stdtypes.html#string-methods
```

Looking Inside Lists

```
friends = [ 'Joseph', 'Glenn', 'Sally' ]
print(friends[1])
```

How Long is a List?

```
greet = 'Hello Bob'
print(len(greet))

x = [ 1, 2, 'joe', 99]
print(len(x))
```

Concatenating Lists Using +

```
a = [1, 2, 3]
b = [4, 5, 6]
c = a + b

print(c)
print(a)
```

Lists Can Be Sliced Using :

```
t = [9, 41, 12, 3, 74, 15]
print(t[1:3])
print(t[:4])
print(t[3:])
print(t[:])
```

Is Something in a List?

```
some = [1, 9, 21, 10, 16]

print(9 in some)
print(15 in some)
print(20 not in some)
# List Methods - 'append', 'count', 'extend', 'index', 'insert',
# 'pop', 'remove', 'reverse', 'sort'
```

Dictionaries

```
cabinet = dict()
cabinet['summer'] = 12
cabinet['fall'] = 3
cabinet['spring'] = 75
print(cabinet)
print(cabinet['fall'])
cabinet['fall'] = cabinet['fall'] + 2
print(cabinet)
```

Comparing Lists and Dictionaries

```
lst = list()
lst.append(21)
lst.append(183)
print(lst)
lst[0] = 23
print(lst)
print()

ddd = dict()
ddd['age'] = 21
ddd['course'] = 182
print(ddd)
ddd['age'] = 23
print(ddd)
```

Dictionary Literals (Constants)

```
jjj = { 'chuck' : 1 , 'fred' : 42, 'jan': 100}
print(jjj)
ooo = { }
print(ooo)
```

Many Counters with a Dictionary

```
ccc = dict()
ccc['csev'] = 1
ccc['cwen'] = 1
print(ccc)

ccc['cwen'] = ccc['cwen'] + 1
print(ccc)
```

Dictionary Tracebacks

```
# It is an error to reference a key which is not in the dictionary
ccc = dict()
print(ccc['csev'])
```

Checking key in dictionary

```
ccc = dict()         # resetting the dictionary
print('csev' in ccc)
```

When We See a New key in dictionary

```
counts = dict()
names = ['Ali', 'Ahmed', 'Ali', 'Zain', 'Ahmed']
for name in names :
  if name not in counts:
    counts[name] = 1
  else :
    counts[name] = counts[name] + 1

print(counts)

name = input("Enter Name: ")
if name in counts:
  x = counts[name]
else :
  x = 0

print(name, "exists", x, "time(s).")
```

The get Method for Dictionaries

```
# The pattern of checking to see if a key is already in a dictionary
# and assigning a default value
# Similar to upper code
names = ['Ali', 'Ahmed', 'Ali', 'Zain', 'Ahmed']
counts = {}
for name in names :
  counts[name] = counts.get(name, 0) + 1


print(counts)

name = input("Enter Name: ")
x = counts.get(name, 0)
print(name, "exists", x, "time(s).")


# Counting words in input
counts = dict()
line = input('Enter a line of text:')

words = line.split() # split() function considers space as delimiter by default;
                  # to use something else as delimiter,
                  # you have to pass that as parameter like line.split(';')
```

```
print('Words:', words)

print('Counting...')
for word in words:
  counts[word] = counts.get(word,0) + 1
print('Counts', counts)
```

```
# Retrieving lists of Keys and Values
dict1 = { 'Ali' : 1 , 'Ahmad' : 42, 'Zain': 100}
print(dict1)
print(list(dict1))
print(list(dict1.keys()))
print(list(dict1.values()))
print(list(dict1.items()))
```

```
# Definite Loops and Dictionaries
counts = { 'Ali' : 1 , 'Ahmad' : 42, 'Zain': 100}
for key in counts:
  print(key, counts[key])
for key,value in counts.items() :
  print(key, value)
```

Activity 4 (Problem Statements / Tasks):

1. *Student Performance Analyzer*

Write a Python program that analyzes students' marks.

Requirements

a) Ask the user to enter the number of students.
b) For each student, input:
   i. Name (string)
   ii. Marks in 3 subjects (integers)
c) Store the data using:
   i. A dictionary where the key is the student name and the value is a list of marks.
d) For each student:
   i. Calculate total marks
   ii. Calculate average marks
   iii. Assign grade using conditional execution:
      - A: average ≥ 85
      - B: average ≥ 70
      - C: average ≥ 50
      - F: otherwise
e) Print:
   i. Student name
   ii. Marks
   iii. Average
   iv. Grade

f) At the end, display:
   i. Class average
   ii. Name of the top-performing student

2. *Comment-Aware Word Frequency Analyzer*

Write a Python program that analyzes text input while ignoring comment lines. A line is considered a comment if it starts with the # character.

Requirements

a) Repeatedly ask the user to enter multiple lines of text.
b) The user should type done to stop entering input.
c) Apply the following rules:
   i. If a line starts with #, ignore it completely using the startswith() function.
   ii. Otherwise:
      - Remove leading and trailing spaces.
      - Convert the line to lowercase. (For case-insensitive counting)
      - Split the line into words.
d) Store and count the frequency of each word using a dictionary. (Use Dictionary .get() method for counting)
e) After input is complete:
   i. Display each word along with its frequency.
f) Ask the user to enter a word and display:
   i. How many times it appears in the text.
g) **(Bonus Marks)** Improve the program so that punctuation does not affect word counting.

3. *Shopping Cart & Billing System*

Design a simple shopping cart system.

Requirements

a) Store items and prices in a dictionary:

```
items = {
 "apple": 100,
 "banana": 50,
 "milk": 120,
 "bread": 80    }
```

b) Repeatedly ask the user to:
   i. Enter item name
   ii. Enter quantity
   iii. Type done to finish
c) If item does not exist, ignore it using continue.
d) Calculate:
   i. Total bill
e) Apply discount rules:
   i. Total ≥ 1000 → 10% discount
   ii. Total ≥ 500 → 5% discount
f) Print:
   i. Original bill
   ii. Discount applied
   iii. Final payable amount

4. *Data Cleaning & Statistical Summary*

You are given a list of sensor readings where -999 represents faulty data.

Requirements

a) Sample list:
   readings = [23, 45, -999, 30, -999, 50, 41]
b) Remove all faulty readings.
c) Calculate and display (using built-in/custom functions):
   i. Cleaned list
   ii. Minimum value
   iii. Maximum value
   iv. Average value
d) Count how many faulty values were removed.

5. *User Login Validation System*

Create a simple login validation system.

Requirements

a) Store usernames and passwords in a dictionary:
   users = {
   "admin": "1234",
   "ali": "pass123",
   "ahmad": "abcd"
   }
b) Allow the user 3 login attempts.
c) For each attempt:
   i. Ask for username and password
d) Validation rules:
   i. If username does not exist → print "Invalid username"
   ii. If password is incorrect → print "Wrong password"
   iii. If correct → print "Login successful" and stop
e) After 3 failed attempts, print "Account locked".